

多范式算子开发实战营 · 第一课

九源软件栈核心技术 与开源生态

统一智能计算架构的设计与实践

王豪杰 | 清华大学 / 启元实验室

AI Infra 开源社区 · GitLink · DataWhale · 启悟学习社区 · 九源社区 · 沐曦股份 联合推出

课程导览

课程内容

从统一架构出发，理解计算、执行与推理系统之间的分工。

- | | | |
|----|--------|-------------|
| 01 | 统一架构 | 模型与硬件怎样解耦 |
| 02 | 算子与编译 | 多种实现怎样统一使用 |
| 03 | 运行时与通信 | 计算资源与设备怎样协作 |
| 04 | 推理框架 | 一次请求怎样完成生成 |
| 05 | 异构推理 | 不同硬件怎样按阶段分工 |
| 06 | 开源共建 | 课程、比赛与项目参与 |

统一接口降低使用差异，异构协作利用性能差异。

第 1 部分

01

统一架构

为什么模型换一种硬件，就需要重新处理软件？

| 平台差异

模型不变，调用环境发生变化

| 统一抽象

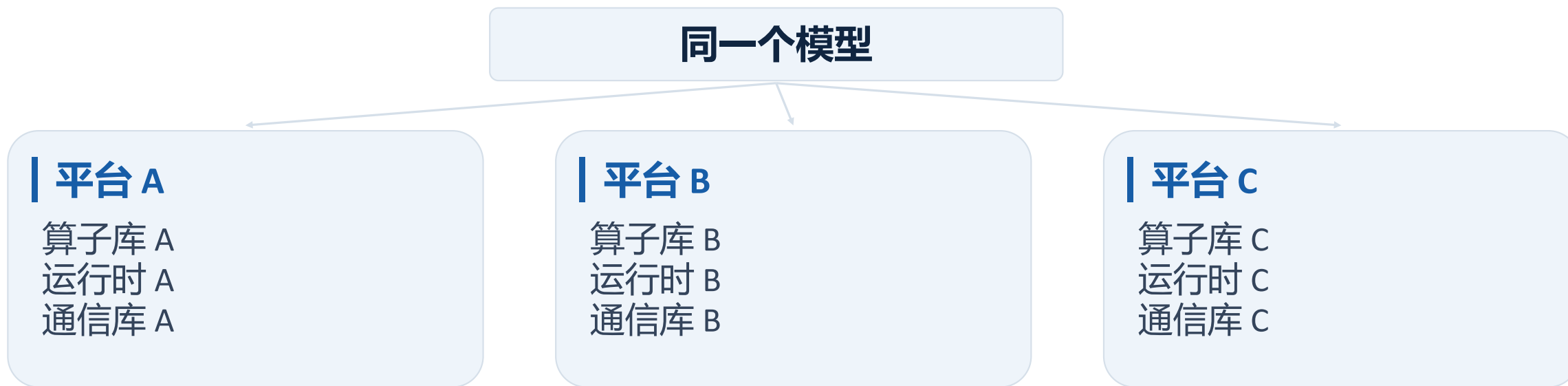
把共性能力集中到基础软件

| 分层设计

框架、基础库与工具各司其职

同一个模型，换一张卡为什么还要改软件？

模型的计算含义没变，底层可调用的能力却发生了变化。



接口 · 执行流程 · 平台协作：哪些部分应该交给统一基础软件？

九源要解决的，是模型与多种硬件之间反复出现的共性问题。

模型运行依赖一整套基础软件

从一次推理请求向下看，计算只是其中一个环节。

模型与应用

决定要完成什么任务



训练 / 推理框架

组织模型执行和请求



算子、运行时、通信

完成计算，管理资源，协同设备



厂商工具链与硬件

把执行请求落实到设备

基础软件的分工

算子：完成张量运算

运行时：管理设备与执行

通信：交换多卡数据

只补齐某个算子或某个框架，并不能自动补齐整条执行路径。

硬件差异不只有峰值算力

计算、访存、容量与互联共同决定实际工作负载的表现。

计算能力

一次能完成多少运算

影响大规模并行计算

内存带宽

单位时间能搬多少数据

影响权重与缓存读取

内存容量

能同时保存多少数据

影响模型、上下文与并发

互联能力

设备之间怎样交换数据

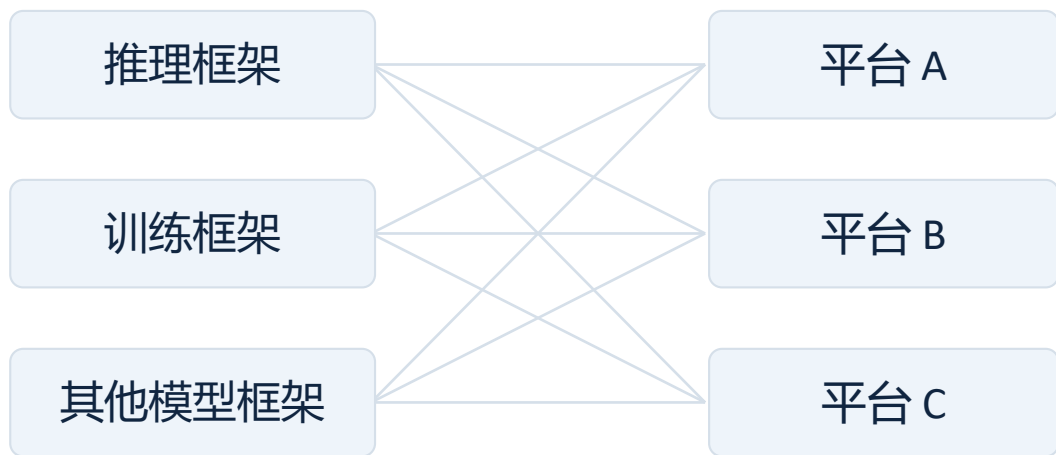
影响多卡与跨节点协作

不同工作负载依赖不同能力，不能只用一个峰值数字判断硬件。

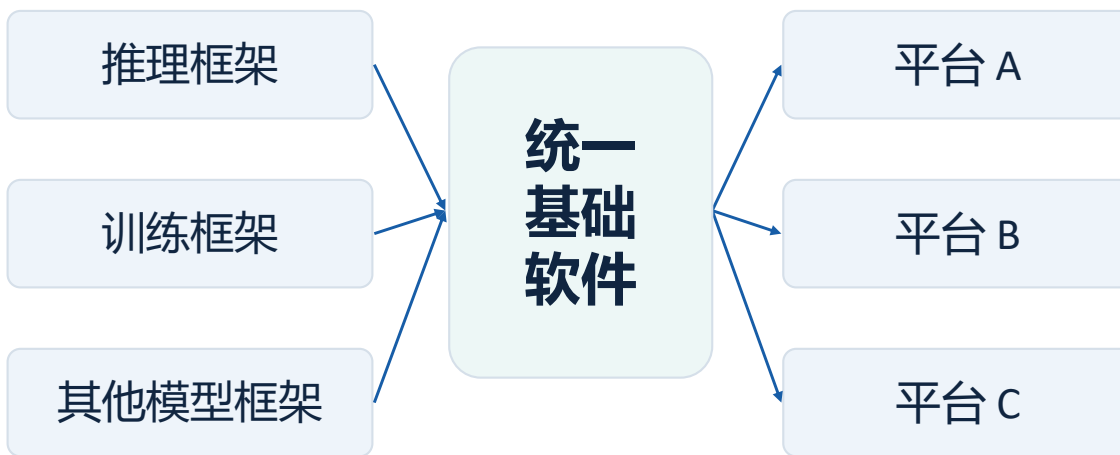
多平台支持中的重复建设

如果每个框架分别对接每个平台，相同工作会在多处重复。

分别建设：每组关系都要维护



公共接口：上层与后端分别扩展



统一架构把共性能力集中维护，让框架扩展与硬件扩展不再相互捆绑。

稳定的底层接口支撑上层创新

借鉴成熟计算生态的分工，而不是把所有功能写进一个框架。

底层需要相对稳定

计算结果与接口含义明确

设备、内存与通信行为一致

能够长期维护和持续优化

上层需要快速变化

模型结构不断更新

训练和推理策略持续变化

应用与服务形态不断扩展

基础软件承担共性支撑，模型与框架才能把精力放在自身创新上。

九源的核心设计目标

围绕国产多样化算力，建设自主可控、可持续演进的基础软件。

统一运行

一套代码面向多种平台

统一抽象与一致性接口

高效执行

利用各平台的原生能力

保留硬件优化与异构协作

持续演进

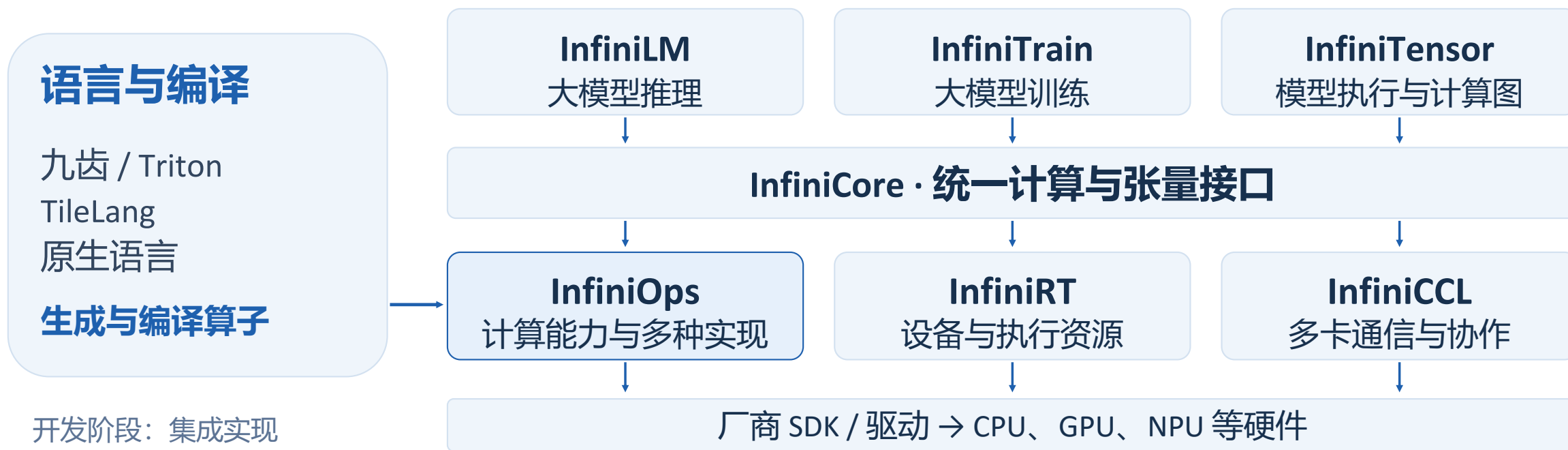
模型、硬件与工具
持续更新

模块化扩展与开源共建

设计目标：一套代码、无需修改、全平台运行，并在系统层面提高算力利用效率。

九源软件栈的分层结构

框架调用统一接口；语言与编译生成的算子集成到 InfiniOps。

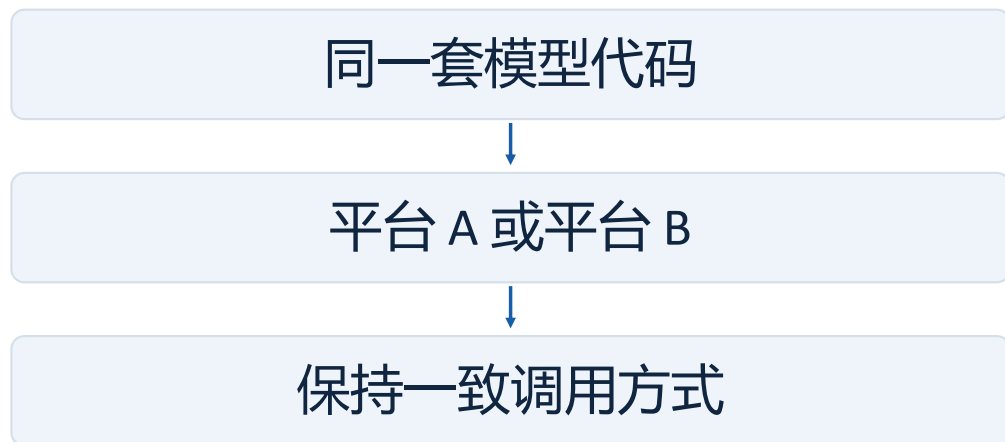


开发侧将实现集成到 InfiniOps；运行侧由框架经统一接口调用。

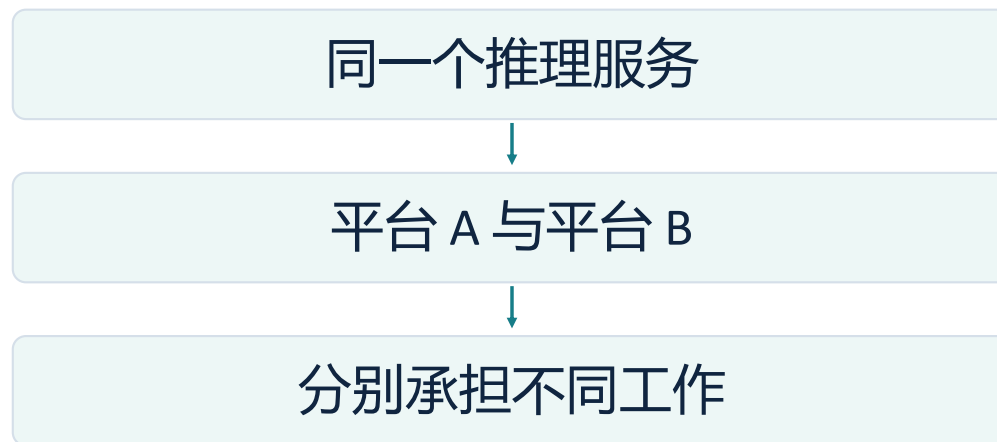
跨平台运行与异构协同是两类目标

前者让代码可以换平台，后者让不同平台可以共同完成任务。

跨平台运行



异构协同



统一接口支持跨平台运行，系统调度进一步组织异构协作。

第 2 部分

02

算子与编译

不同硬件、不同库、不同语言，怎样供框架统一使用？

| 统一算子库

处理语义、接口和调用差异

| 九齿与编译

数据编排与串行语义

| 实现集成

编译产物进入 InfiniOps

算子库连接模型计算与硬件执行

把反复出现的张量运算组织成可复用的计算能力。

模型层示例

前馈网络

注意力层

归一化层

常用算子

矩阵乘

Attention

归一化 / 激活

实现来源

原生算子库

已有开源实现

DSL / 自定义实现

模型表达需要什么计算，算子库负责提供合适的执行实现。

相同的计算，可能面对不同的调用方式

统一算子库要同时处理三类差异。

硬件平台不同

接口名称、行为与支持范围不同

不能只替换设备名称

算子库来源不同

参数表达、资源准备和调用顺序不同

不能只替换一个函数

开发工具不同

DSL 的编译产物与运行依赖不同

不能直接把脚本交给框架

计算能力相同，不代表上层调用方式相同。

硬件平台之间，接口和行为都可能不同

相似的功能名，不一定意味着相同的使用约定。

差异位置	上层可能遇到的问题	统一层需要处理
计算接口	输入表达、数值类型和布局不同	定义一致的计算含义
执行资源	设备、内存和任务提交方式不同	与统一运行时协同
能力覆盖	支持的算子和输入范围不同	明确能力范围与扩展入口

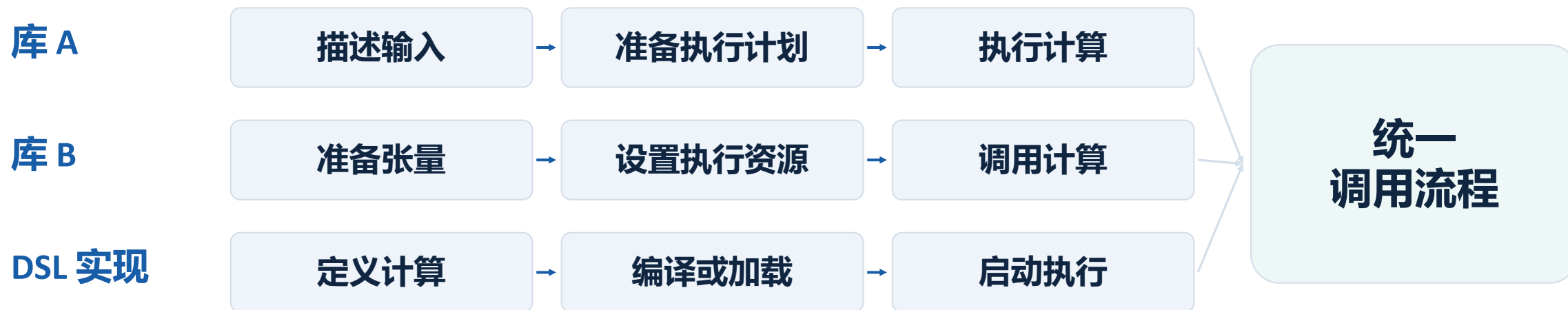
平台差异应在后端被处理，而不是在每个模型中反复判断。

MODULE 02 · 算子与编译

同一平台上，不同算子库也不容易互换

已有库的调用流程不同，框架不宜分别承担这些差异。

常见调用模式示意



统一库吸收不同调用流程，让上层按相同方式使用计算能力。

DSL 算子与框架之间还缺一调用衔接

写出计算表达，与把它交给推理框架使用，是两件事。

DSL 开发

- DSL 源程序
- 编译、调优与缓存
- 依赖相应开发环境

InfiniOps 内衔接

- 可调用的编译产物
- 张量与执行资源衔接
- 加载、错误与生命周期管理

框架运行

- 框架持续调用
- 复用统一资源
- 与其他算子共同执行

DSL 负责表达和生成计算，InfiniOps 负责集成与统一调用。

统一算子库：统一语义、接口与调用流程

对上保持一致，对下保留不同实现的优化空间。

| 计算语义

输入输出含义一致
支持范围明确

| 调用接口

上层使用统一入口
不区分实现来源

| 调用流程

资源准备与执行衔接
遵循一致的使用方式

多种实现保持独立优化

厂商优化库

原生语言实现

DSL 编译产物

统一的是上层使用方式，不是要求所有平台采用同一套底层代码。

多种实现通过统一入口供框架使用

接入已有能力与开发新算子，可以采用同一套组织方式。



实现进入 InfiniOps，上层通过 InfiniCore 等统一接口使用计算能力。

保留原生实现，才能利用硬件已有的优化

统一抽象不应成为抹平性能差异的理由。

成熟能力优先复用

保留厂商优化过的计算路径

沿用适合该设备的算法

减少不必要的数据转换

新需求仍可独立扩展

补充缺失或专用的算子

引入融合和新的开发范式

按具体负载改进实现

接口保持统一，性能优化留在最了解硬件和负载的位置。

算子库还要解决选优、扩展与裁剪

建设内容包括按负载选优、扩展新计算，以及按部署需求裁剪。

选优

不同输入适合不同实现

按负载与设备选择计算路径

扩展

新模型带来新的计算需求

沿公共接口增加实现

裁剪

部署环境的资源约束不同

只保留需要的能力与依赖

统一算子库的建设目标，不止是积累更多算子源文件。

为什么在统一架构中设计编程语言？

已有算子库覆盖常见计算，但模型创新仍会提出新的计算需求。

只组合已有算子

开发方便

遇到新计算或融合需求时受限

直接使用底层语言

控制细致

需要更多并行与硬件知识

张量级领域语言

保留定制能力

降低描述计算与组织并行的门槛
一套代码编写的算子可以运行在不同平台上

九齿希望让模型开发者也能参与高性能计算实现，而不必从全部底层细节学起。

九齿：先编排数据，再描述局部计算

以矩阵乘 $C = A \times B$ 为例：每个任务计算 C 的一个输出块。



沿 k 取出对应的两个分块，求乘积，再累加到同一个输出块。

arrangement 确定任务与数据的关系；application 描述每个任务的运算。

九齿示例：用串行语义描述矩阵乘

application 接收编排后的数据；下面只展示一个输出块的计算。

矩阵乘的 application

```
import ninetoothed.language as ntl

def application(input, other, output):
    acc = ntl.zeros(output.shape,
                    dtype=ntl.float32)
    for k in range(input.shape[0]):
        acc += ntl.dot(input[k], other[k])
    output = acc
```

01 初始化

为当前输出块建立累加器

02 顺序累加

k 遍历该任务的输入分块

03 写回结果

九齿将赋值编译为输出写回

先置零，再沿 k 累加，最后写回；代码表达的是局部算法顺序。

串行的计算表达，怎样变成并行执行？

不同输出块可以独立处理；每个任务复用同一段 application。

输出矩阵 C 的任务划分

C[0, 0]

独立任务

C[0, 1]

独立任务

C[0, 2]

独立任务

C[1, 0]

独立任务

C[1, 1]

独立任务

C[1, 2]

独立任务

同一份局部代码，作用于不同的数据块。

不同输出块之间

可独立处理，由编译器映射到并行程序。

每个输出块内部

按 k 累加对应分块，保留计算依赖。

开发者与编译器分工

开发者描述编排与计算；编译器组织设备执行。

。

块间并行来自数据编排；块内顺序来自计算依赖；编译器完成设备映射。

多范式开发可以共存，不必选择唯一语言

不同工具面向不同层次的控制需求，最终都服务于模型执行。

开发方式	主要关注点	在统一软件栈中的位置
原生语言 / MACA C++	较细的硬件与并行控制	提供平台相关实现
Triton / TileLang	块状计算与数据组织	提供领域专用实现
九齿	张量编排与串行式计算表达	降低定制计算的开发门槛

多范式提供不同的计算表达，InfiniOps 提供统一的使用方式。

编译生成实现，InfiniOps 组织集成与调用

开发工具和框架通过算子库衔接，各自保留独立的演进空间。

开发阶段：表达计算并形成目标设备实现



运行阶段：框架通过统一接口调用已集成的算子



库内集成

AOT 可以提前准备产物；InfiniOps 负责将实现接入统一调用路径。

第3部分

03

运行时与通信

有了计算能力，还需要怎样管理资源与多卡协作？

资源管理

设备、内存与任务依赖

统一通信

以一致接口组织多卡数据交换

协同执行

计算、运行时与通信相互配合

只有算子库，模型仍然不能完整运行

InfiniRT 统一管理设备选择、数据存放与执行依赖。

执行之前

设备和数据在哪里？

算子计算

执行之后

资源何时可以再次使用？

InfiniRT · 统一硬件运行时

执行过程中

任务之间按什么顺序？

把计算调用统一起来之后，还要把计算所依赖的资源与执行过程统一起来。

InfiniRT：统一管理资源与执行关系

既要让任务执行正确，也要减少重复准备和无效等待。



权重

跨请求保留

请求缓存

随请求保留和增长

临时数据

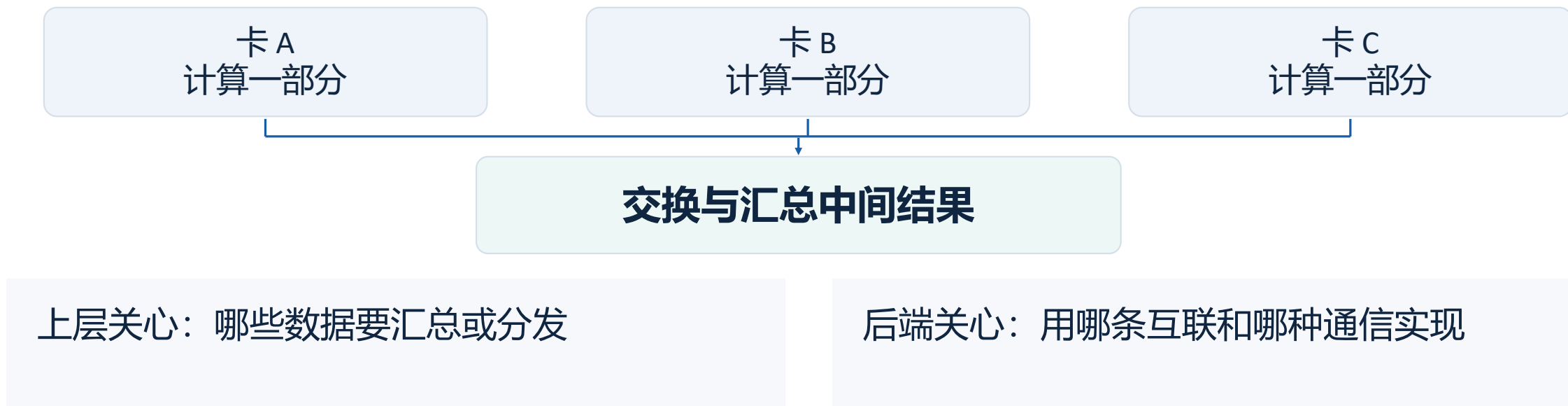


按执行阶段复用，避免反复申请和无效等待

资源何时可用、任务何时完成，需要由一致的执行关系来约束。

InfiniCCL：统一多卡数据交换

模型分布到多张卡以后，计算结果必须按约定交换。



统一通信接口，让模型并行策略不必直接绑定某一家的通信库。

统一通信接口之下可以有不同传输路径

InfiniCCL 在一致接口下，结合硬件支持选择不同通信后端。

统一通信接口 · 表达数据交换需求

| 原生集合通信

利用同构环境已有优化

| MPI 等通用路径

支撑更广泛的部署环境

| RDMA 等传输机制

降低跨节点数据移动开销

结合硬件支持、互联拓扑与数据规模组织通信

统一的是通信语义，不是要求所有环境使用同一种传输实现。

换硬件时，哪些工作应该复用？

用三个变化，检查统一架构是否真正把职责分开。

发生的变化	优先修改的位置	应该尽量复用
新增一种硬件	算子、运行时与通信后端	模型结构与框架逻辑
增加一种算子实现	编译与 InfiniOps 内实现选择	公共计算接口
改进请求调度策略	推理框架和资源组织	已有底层计算能力

好的分层让变化主要留在对应组件，而不是扩散到整个系统。

第 4 部分

04

推理框架

怎样把底层计算组织成持续可用的推理服务？

| InfiniLM

管理模型执行与请求状态

| 两阶段推理

Prefill 与 Decode 的不同负载

| 服务指标

区分等待时间、生成间隔与吞吐

统一基础软件之上，为什么还需要 InfiniLM?

有了计算能力，还需要把它组织成持续可用的推理服务。

基础软件解决

单个计算怎样完成

资源怎样统一使用

多卡数据怎样交换

InfiniLM 解决

请求怎样进入与排队

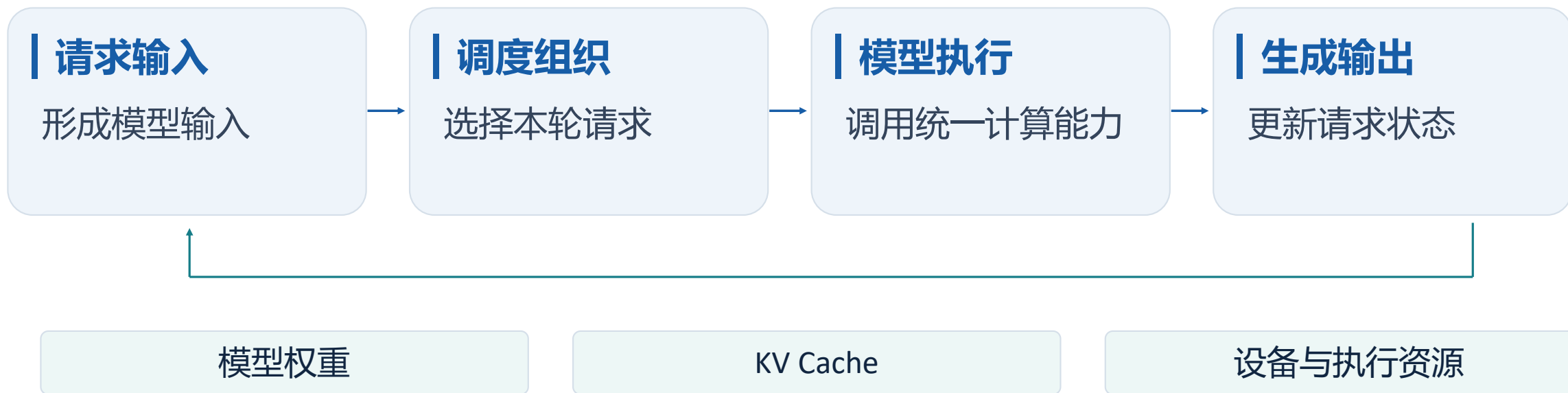
模型和缓存怎样持续执行

计算资源怎样服务多个请求

统一基础能力与跨平台推理框架配合，系统优化才能在不同硬件上复用。

InfiniLM 怎样组织一次推理请求？

请求处理、模型执行和缓存管理共同构成推理过程。



请求状态贯穿多轮执行，推理引擎不只是把算子顺序调用一遍。

一次生成包含 Prefill 和 Decode 两个阶段

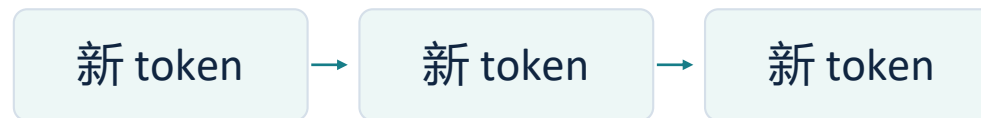
先处理提示词，再利用已有上下文逐步生成。

Prefill (P)



处理提示词
建立上下文缓存

Decode (D)



读取已有上下文
逐步生成后续 token

两阶段使用同一个模型，但每轮处理的数据规模与访问方式不同。

P 与 D 的差异，来自计算和数据访问方式

批量计算中的数据复用，与逐步生成中的反复读取，形成不同瓶颈。

Prefill：一次处理较多 token



同一组权重参与较多计算

更容易形成较大的矩阵运算

通常更依赖计算吞吐

Decode：每轮新增少量 token



权重
历史 KV

持续读取权重与历史 KV

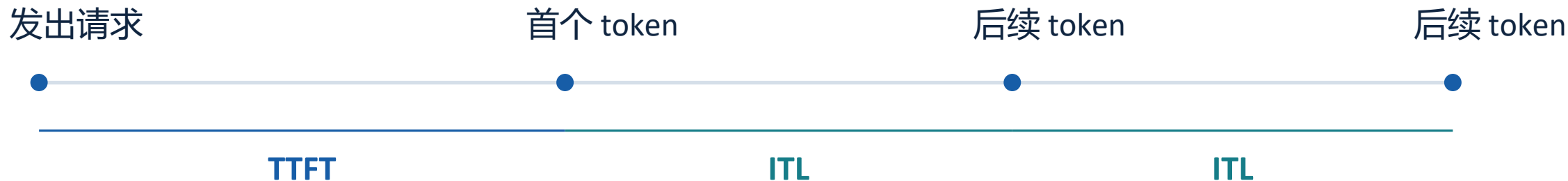
每轮新增计算相对较少

通常对带宽与执行时延更敏感

这是典型负载特征；模型、上下文长度和并发会改变具体瓶颈。

推理性能需要同时看等待、生成与吞吐

先看用户等待的是什么，再选择相应的评价指标。



TTFT

首个 token 的等待时间

ITL

相邻输出 token 的时间间隔

吞吐

单位时间处理或生成的 token 数

在线交互与离线批处理，对时延、吞吐和资源成本的侧重不同。

混合执行为什么会产生阶段间干扰？

长提示词的处理和已有请求的逐步生成，可能竞争同一批资源。

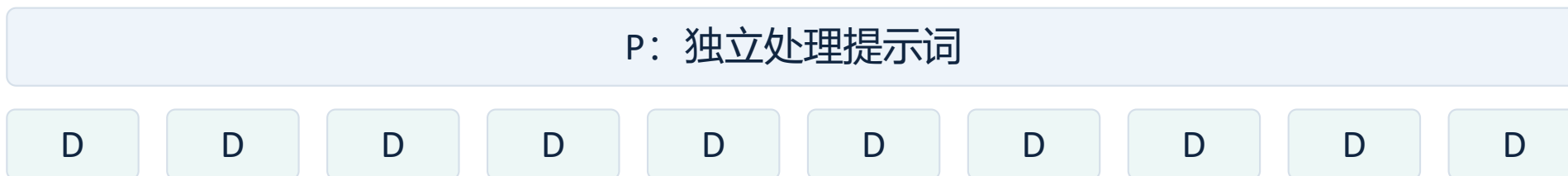
同一批资源：提示词处理与生成交替执行

混合执行



生成间隔可能被拉长

独立资源



PD 分离首先改变执行组织，让不同阶段能够独立排队和安排批处理。

第 5 部分

05

异构推理

P 与 D 需要的能力不同，能否让不同硬件分别承担？

| 阶段分工

按实际负载选择 P/D 资源

| 状态交接

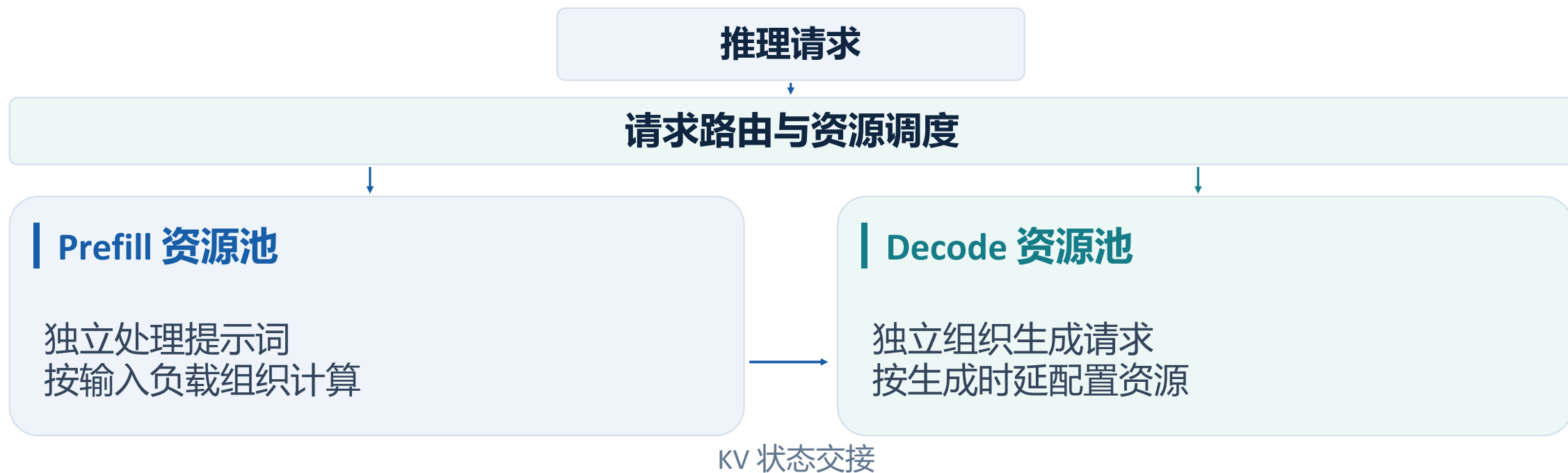
通过 KV 延续同一请求

| 系统收益

结合传输、配比与实验分析

PD 分离：让两个阶段分别组织资源

两侧都能执行完整模型；分离的是推理阶段，而不是按模型层切分。



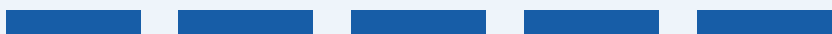
两阶段可以分别扩展，也可以选择不同类型的硬件。

不同硬件的长处，可以匹配不同阶段

两阶段需求不同，硬件的优势也不一定相同。

| Prefill 资源

计算能力更合适的资源



优先考虑 Prefill

| Decode 资源

带宽与缓存能力更合适的资源



优先考虑 Decode

同时检查：阶段实际性能 · 容量与并发要求 · 互联和使用成本

阶段优势互补且交接成本可控时，异构分工可以提高整体执行效率。

按阶段选卡，而不是只看整卡排名

教学假设：固定请求、同一对卡；数字表示相对耗时。

硬件	完成 P 阶段	完成 D 阶段
卡 A	2	8
卡 B	6	3

$$A \text{ 做 } P \rightarrow B \text{ 做 } D: 2 + 1 + 3 = 6$$

$$B \text{ 做 } P \rightarrow A \text{ 做 } D: 6 + 1 + 8 = 15$$

假设交接均耗时 1 个单位；仅说明分工原则，不代表实测性能或吞吐。

PD 分离后，为什么必须传递 KV Cache？

D 要延续同一个请求，就需要 P 已经建立的上下文状态。

P：建立上下文



D：延续生成



请求关联 + KV 数据交接

KV Cache 是请求的上下文状态，不是模型权重；交接完成后才能正确续写。

异构 PD 中，各层分别负责什么？

请求调度、统一计算、设备资源和状态传输需要共同配合。

分工	主要组件	承担的工作
请求与阶段组织	InfiniLM	组织 P/D 执行与请求状态
统一计算与资源	InfiniCore / InfiniOps InfiniRT	提供统一张量调用、算子实现 与设备资源管理
模型内部通信	InfiniCCL	实例采用模型并行时 交换和汇总数据
跨阶段状态交接	KV Connector / Mooncake	将 P 形成的有效 KV 交接给 D 继续生成

模型内部的多卡通信与 P/D 之间的 KV 交接，是两类不同的协作。

分离后的收益取决于计算、传输与资源配比

阶段分工只是起点，整体效率还受最慢环节和排队影响。

| 阶段性能

P 与 D 的资源是否匹配

避免某个阶段长期积压

| 状态传输

KV 能否及时交接

减少搬运与数据整理开销

| 资源配比

两个资源池的
处理能力是否平衡

按负载调整资源配比

以服务指标验证分离收益，不能只把两个阶段的单独跑分相加。

异构 PD 提供了缩小应用性能差距的另一条路径

除了提升单卡能力，还可以改变系统使用已有硬件的方式。

逐卡承担完整流程

一张卡同时兼顾 P 和 D

短板可能限制整个请求

应用受单个平台软件能力约束

按阶段发挥硬件特长

匹配计算与访存优势

在系统层面组织互补

提高既有资源的有效 Token 产出

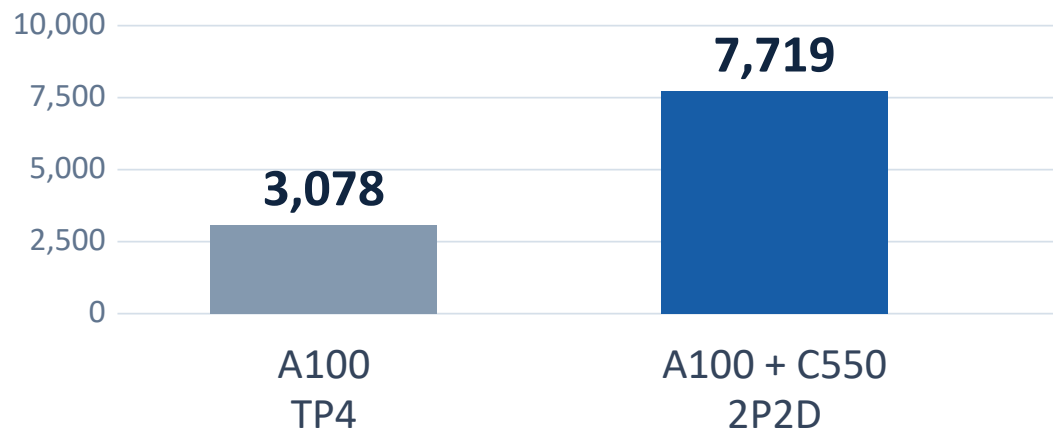
九源既解决多平台使用门槛，也探索通过异构协作缩小实际应用性能差距。

MODULE 05 · 异构推理

实验一：A100 与 C550 的异构 PD 组合

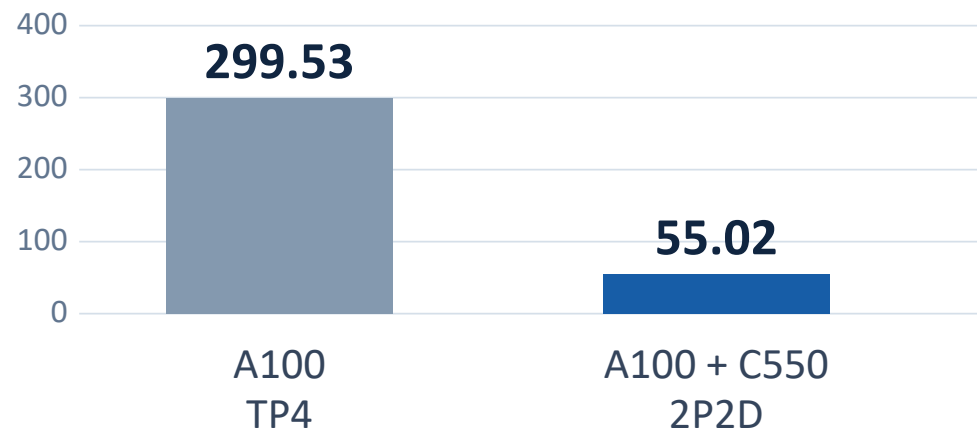
比较 4 卡 A100 张量并行与 A100 + C550 的 2P2D 实验记录。

输出吞吐 (tok/s) ↑



输出吞吐约为 2.5 倍

平均 ITL (ms) ↓



ITL 降至约 1/5.4

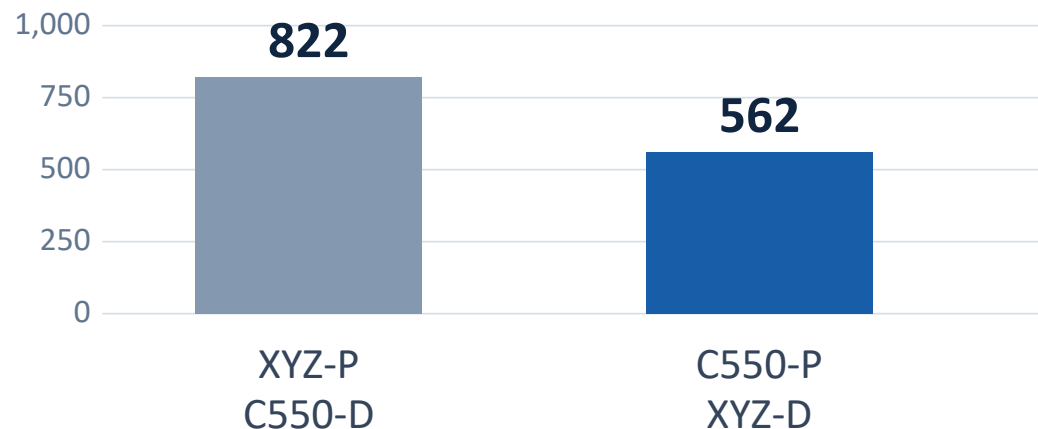
该组实验卡数相同，卡型与执行方式不同；结果反映组合方案，不等同于单卡对比。

MODULE 05 · 异构推理

实验二：同一组硬件，交换 P/D 角色

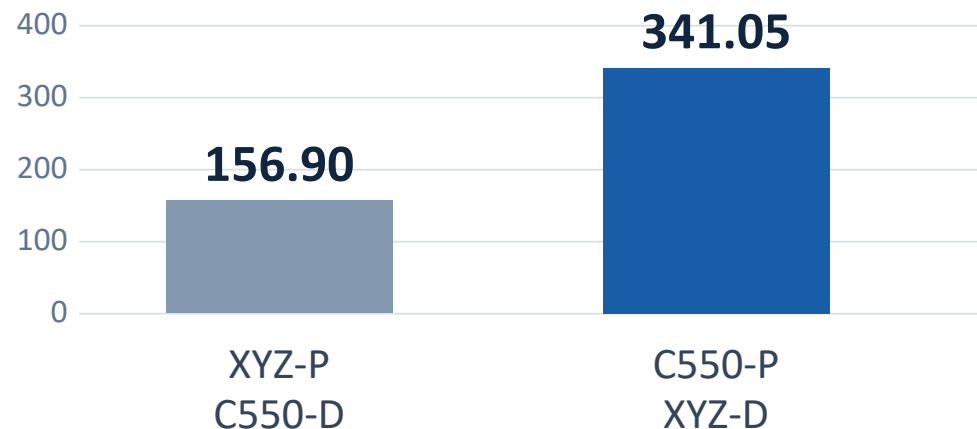
XYZ 与 C550 的两种阶段分工，产生了不同的性能结果。

输出吞吐 (tok/s) ↑



哪一种分工输出更快？

平均 ITL (ms) ↓



哪一种分工生成更连贯？

比较的是硬件与阶段的匹配关系，而不只是卡的名称或数量。

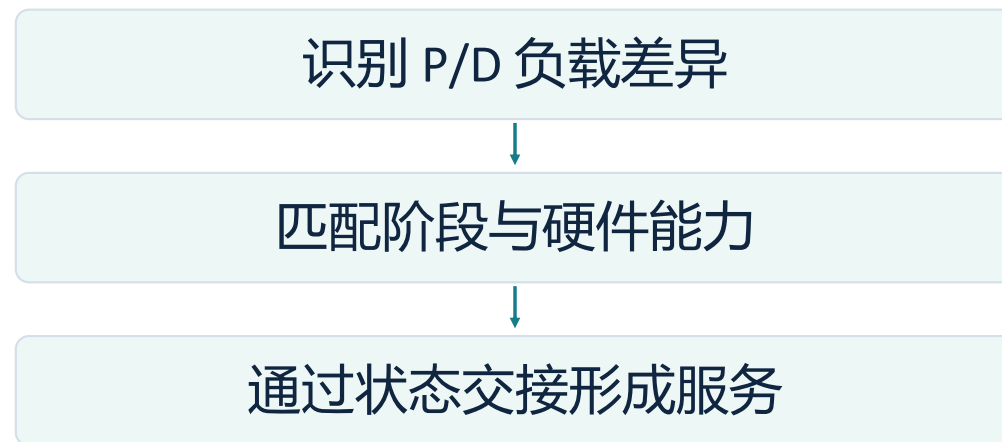
从跨平台运行到按特长协作

统一架构与异构优化分别回答两层问题。

先解决共同使用



再解决有效分工



统一抽象减少使用差异，异构调度利用性能差异；两者并不矛盾。

第 6 部分

06

开源共建

课程与比赛中的实践，怎样进入可持续维护的开源项目？

课程学习

理解人工智能系统各层原理

比赛实践

围绕真实项目完成改进

开源贡献

通过测试、文档和代码参与

九源开源项目：从一个具体问题开始参与

语言、基础库与框架对应不同任务，可以按兴趣选择入口。

语言与算子

[九齿 / ntops](#)
[InfiniOps](#)

编程表达与编译用例
算子实现与跨平台测试

运行与通信

[InfiniCore / InfiniRT](#)
[InfiniCCL](#)

张量接口与资源管理
多卡协作与问题复现

模型与框架

[InfiniLM / InfiniTrain](#)
[InfiniTensor](#)

模型执行与系统优化
训练推理用例与文档

选择一个 Issue



复现与定位



修改并补测试



提交 PR

复现问题、补充测试、修正文档或改进实现，都可以成为开源贡献。

MODULE 06 · 开源共建

课程学习：从基础知识进入系统实践

启元“InfiniTensor：大模型与人工智能系统训练营”

基础学习

C++ / Python
并行编程与硬件基础

方向课程

算子与编译
训练、推理与通信

项目实践

命题或自主选题
进入真实开源项目

用“小系统”理解整体，再进入完整项目

AI 编译器

TinyInfiniTensor

大模型推理

LlaiSys

训练系统

TinyInfiniTrain

课件、视频回放、方向作业与项目实践可以配合使用，按兴趣选择学习方向。

比赛实践：围绕真实项目完成改进

启元人工智能大赛以开源项目为基础，提供算子、编译和训推系统实践方向。

赛题方向示例

九齿与编译

九齿 / ntops

算子性能优化
编译能力与测试改进

训练与推理

InfiniLM / InfiniTrain

模型执行与性能分析
训练系统核心功能

智能编程

九齿.skill

AI 辅助开发
调试与自动优化

了解赛题与基线 → 组队开展实践 → 提交可复现结果 → 继续参与项目维护

把实现、测试和复现记录一起整理，比赛成果才能继续在开源项目中发挥作用。

持续建设：沿共同接口扩展能力

完善 InfiniOps、运行时、通信与编译，并通过模型场景持续验证。

建设方向	需要继续解决的问题	预期作用
统一基础库	完善算子、后端和跨平台测试	扩大统一运行范围
语言与编译	多目标支持、产物交付与实现选优	提高开发与执行效率
异构推理	KV 组织、传输与资源配比	提高复杂场景系统效率
开源协作	课程、文档、评测与共同维护	形成持续演进能力

建设主线保持一致：统一使用方式，保留优化空间，支持更多模型与硬件。

课程回顾：各层分别解决什么问题？

把后续的硬件、语言、算子与性能课程放回同一套设计中理解。

为什么统一算子库？

处理平台、库和 DSL 的调用差异

为什么配合运行时与通信？

计算必须使用资源，也需要多卡协作

为什么采用异构 PD？

按阶段需求发挥不同硬件的长处

硬件与 MXMACA

理解资源能力

九齿 / TileLang / 算子课程

完成计算实现

性能分析与项目实践

验证模型与系统收益

学习实现方法时，同时关注它在统一软件栈中的位置，以及它解决了什么问题。

课程交流

继续学习，参与开源实践

课程学习 · 比赛实践 · 开源项目贡献



训练营项目与学习社群

gitlink.org.cn/ccf-ai-infra/gollamago

九源社区: infinitemensor.com

从一个用例、一项测试或一次代码改进开始。